<table>
<tr><td colspan="2">Nombre:

José Uriel Hernández Hdez

Carlos Tomas Cárdenas Vázquez

Pamela López Martínez

Jaime Guerra Jr</td><td>Matrícula:

2226704

2226997

2226900

2227428</td></tr>
<tr><td>Nombre del curso: Componentes de lógica de negocio</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Modulo Netacad: 3</td><td colspan="2">Actividad: UNIDAD 3</td></tr>
<tr><td colspan="3">Fecha: 15 de octubre del 2026</td></tr>
<tr><td colspan="3">Bibliografia (Formato APA)</td></tr>
</table>

Resumen Ejecutivo

Eventos Nova es una empresa dedicada a la organización de bodas, graduaciones y eventos empresariales, cuyos ingresos provienen de la coordinación y contratación de servicios con proveedores. Actualmente, la operación presenta fallas críticas de comunicación: los cambios solicitados vía WhatsApp no llegan a todos los involucrados, generando versiones distintas de la propuesta y ocasionando que los proveedores reciban información desactualizada. Además, existe dificultad para controlar los costos y cambios autorizados.

El objetivo de este proyecto es desarrollar un sistema centralizado que gestione el proceso de cotización, contratación, coordinación y ejecución. Para lograrlo, el presente informe detalla el diseño arquitectónico de software que soportará dichas operaciones, asegurando la consistencia de los datos y el control estricto de las reglas de negocio.

Objetivo del proyecto.

El éxito de la implementación de esta arquitectura y sistema se medirá a través de los siguientes criterios:

- Obtener una calificación aprobatoria consistente en la encuesta de satisfacción enviada a los clientes.
- Reducir a cero las discrepancias financieras y operativas causadas por información desactualizada.
- Generar exitosamente reportes mensuales administrativos que reflejen métricas clave (eventos exitosos, ganancias, y proveedores más utilizados).

Actores del sistema.

Para el correcto funcionamiento del sistema, se han identificado cuatro perfiles principales de interacción (Stakeholders), cada uno con responsabilidades y alcances bien definidos dentro de la plataforma:

- **Cliente:** Actor que proporciona los requerimientos iniciales, posee acceso en modo de solo lectura para consultar la oferta de servicios, y es responsable de aprobar propuestas y realizar pagos.
- **Coordinador:** Actor central encargado de la atención directa al cliente. Levanta los requerimientos iniciales, elabora y aprueba cotizaciones, gestiona los cambios operativos y sirve de enlace directo con los proveedores.
- **Administración:** Usuario con privilegios exclusivos (RBAC) encargado de gestionar el catálogo de proveedores, actualizar los precios, validar financieramente los anticipos, aplicar penalizaciones y generar reportes ejecutivos.
- **Proveedor:** Entidad externa contratada para prestar servicios específicos (salón, música, fotografía, etc.). Interactúa con el sistema confirmando su disponibilidad y recibiendo notificaciones automáticas.

Etapa 1. Preparar el caso.

Para el diseño detallado de la arquitectura de software, se han delimitado cinco casos de uso críticos extraídos de las historias de usuario originales. Esta selección abarca intencionalmente tanto flujos normales (caminos de éxito) como flujos de excepción y rechazo, asegurando que la arquitectura propuesta tenga la capacidad de manejar las restricciones financieras y temporales más exigentes de Eventos Nova:

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Aprobación de cotización (US-04)	Coordinador	Generar folio, estado "Pendiente", calcular 25% de anticipo.	Propuesta cambia a "Enviada" y se calcula el anticipo automático.
CU-02	Cancelación y lista de advertencia (US-07)	Sistema	Detectar cancelación de proveedor, sugerir reemplazo, clasificar.	Alerta enviada al Coordinador y proveedor etiquetado en lista negra
CU-03	Limite de cambios normales (US-08)	Coordinador	Límite estricto de 10 días previos al evento para modificaciones.	Bloqueo automático de modificación; exige autorización especial
CU-04	Bloqueo de presupuesto (US-11)	Sistema	Bloquear servicios adicionales que excedan presupuesto inicial.	Botón de guardar desactivado y muestra error por límite financiero.
CU-05	Penalización por saldo pendiente (US-13	Administrador	Tolerancia de 2 días post-fecha de liquidación, cancelación en cascada.	Servicios marcados para cancelación, aplicación de cobro extra.

Es importante destacar que los cinco casos de uso detallados en la tabla anterior fueron extraídos de manera directa e íntegra de las Historias de Usuario (formato Gherkin), el Documento de Requisitos Empresariales (BRD) y el diagrama BPMN aprobados previamente en la Unidad 2. El equipo de desarrollo mantuvo intacto el alcance original y no inventó nuevas reglas de negocio para facilitar o acomodar el diseño arquitectónico. Cada flujo operativo, restricción financiera y validación de tiempo cuenta con una trazabilidad directa hacia el modelo original, garantizando que la arquitectura que se construirá a continuación resolverá los escenarios reales y las fallas críticas de comunicación de Eventos Nova.

Etapa 2. Elegir el patrón de lógica de negocio.

Para establecer los cimientos de la arquitectura del sistema, el equipo evaluó tres patrones fundamentales de lógica de negocio: *Transaction Script*, *Table Module* y *Domain Model*. La evaluación no se basó únicamente en tendencias tecnológicas, sino en el comportamiento real exigido por Eventos Nova, analizando la cantidad de reglas interconectadas, los cambios esperados, el manejo de estados (como el vencimiento de tiempos) y la obligación de proteger los invariantes financieros y operativos. A continuación, se presenta la matriz comparativa con el impacto de cada alternativa en nuestro contexto:

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Pobre. Mezcla validaciones de negocio con sentencias de base de datos.	Medio. Útil para CRUDs, difícil al aplicar reglas entre múltiples entidades cruzadas.	**Excelente.** Aísla reglas complejas (límites de tiempo, penalizaciones) en objetos propios	BRD (Restricciones) y US 08, 11, 13.
Cambios esperados	Difícil. Alterar reglas de anticipo obliga a buscar y editar múltiples scripts.	Moderado. Estructura altamente acoplada al esquema de la base de datos.	**Alto.** Modificar lógicas de cancelación o presupuesto no afecta la capa de persistencia	BRD (Necesidades del Negocio).
Interacción entre conceptos	Riesgo de código espagueti al cruzar Cliente, Contrato y Proveedor en un solo script.	Requiere manejar la lógica mediante múltiples RecordSets asociados.	**Fluida.** Los objetos (Aggregates) controlan y validan sus propias dependencias internas	Modelo de Dominio y BPMN.
Estados e invariantes	Nulo. No protege transiciones de estado de forma nativa.	Delega la protección de invariantes a restricciones puramente de la BD (SQL).	**Ideal.** Protege invariantes críticos como "Contrato Activo" o "Expirado" directamente en la clase	Glosario (Definición de Contrato Activo).

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Criterio adicional	Los cálculos de fechas y penalizaciones se programan directamente en funciones sueltas, lo que genera duplicidad y riesgo de errores si las reglas cambian.	La validación se hace a nivel de tabla, pero se complica cuando el límite de tiempo depende del estado de otro registro (ej. fecha de Contrato vs fecha de Evento).	**Excelente.** Los objetos manejan su propio ciclo de vida. Una entidad Contrato calcula internamente si aún está en el periodo de gracia o si ya aplica penalización.	BRD (Restricciones temporales y financieras) y Glosario (Límite de Cambios Normales)

Decisión argumentada:

Tras analizar la complejidad operativa de Eventos Nova y evaluar los patrones Transaction Script, Table Module y Domain Model, en la tabla comparativa anterior. El equipo ha decidido implementar **Domain Model** como la arquitectura central de lógica de negocio. Esta decisión no se basa únicamente en la escalabilidad o modernidad del patrón, sino en la necesidad estricta de manejar los múltiples estados, transiciones y reglas financieras que se establecieron anteriormente para el negocio.

La justificación por la cual se eligió ese modelo radica en los siguientes problemas específicos extraídos del BRD de la unidad 2.

Protección de Invariantes Financieros y Presupuestales: Eventos Nova sufre actualmente por una fuerte dificultad para controlar los costos y los cambios autorizados. Una regla de negocio estricta (US 11) dicta que el sistema debe bloquear automáticamente los servicios adicionales si estos superan el presupuesto inicial aprobado. Usar un Transaction Script dejaría esta validación expuesta en la capa de interfaz o en funciones procedimentales dispersas. Con Domain Model, el agregado Contrato o Evento se convierte en el único responsable de autorizar adiciones. El modelo protege el invariante impidiendo que la lista de servicios crezca si el CostoTotal excede el PresupuestoAprobado, lanzando una excepción de dominio antes de siquiera tocar la base de datos.

Gestión de Reglas Temporales Complejas: La operación de Nova depende de límites de tiempo: los proveedores tienen de 2 a 3 días para confirmar; los clientes tienen hasta 10 días previos al evento para solicitar cambios de menú o cancelaciones; y existen penalizaciones por saldos pendientes tras 2 días de periodo de gracia. Un Table Module o Transaction Script obligaría a crear scripts de bases de datos o rutinas externas muy pesadas para calcular estas fechas en cada consulta. El Domain Model resuelve esto encapsulando la lógica temporal. El objeto Evento conoce su propia fecha de ejecución y posee métodos como SolicitarCambio(), el cual internamente valida si la diferencia entre la fecha actual y la fecha del evento es mayor a 10 días. Si no lo es, el objeto mismo rechaza la modificación y protege la integridad del estado.

Transiciones de Estado de Proveedores (Lista de Advertencia): Cuando un proveedor cancela de forma repentina, no solo se debe actualizar la base de datos, sino que el sistema debe desencadenar sugerencias de reemplazo y mover al proveedor a una "lista de advertencia" (US 07). El Domain Model facilita la emisión de Domain Events (Eventos de Dominio). Cuando la entidad Proveedor ejecuta el método CancelarParticipacion(), dispara un evento que la capa de aplicación escucha para notificar al Coordinador, logrando una arquitectura reactiva y sin acoplamiento duro.

Costos técnicos y desventajas introducidas

La elección de Domain Model no está exenta de sacrificios. Al elegir este patrón, el equipo acepta las siguientes consecuencias y costos para el proyecto:

- **Curva de aprendizaje:** A diferencia de un simple CRUD (Create, Read, Update, Delete) manejado por Transaction Script, el Domain Model exige crear una gran cantidad de código inicial (clases, interfaces, repositorios, value objects y mapeadores) antes de poder ver una funcionalidad operativa.
- **Complejidad en el Mapeo de Datos:** Dado que la estructura de los objetos de negocio no coincidirá exactamente con las tablas de la base de datos relacional, se introduce la necesidad imperativa de utilizar un ORM (Object-Relational Mapper) avanzado o mapeadores manuales (Patrón Repository) para traducir los objetos de dominio a filas de base de datos. Esto añade sobrecarga de procesamiento.
- **Problemas de Rendimiento en Lecturas Masivas:** Cargar un Aggregate completo (ej. un Evento con sus 20 proveedores y su contrato) a la memoria solo para leer el nombre del cliente resulta ineficiente y costoso en términos de RAM y tiempo de procesamiento.
- **Mitigación:** Para combatir este costo técnico, la arquitectura se apoyará en la **Etapa 6 (Separación de Commands y Queries - CQRS)**, delegando las escrituras y validaciones complejas al Domain Model, pero permitiendo que las consultas de lectura simples ataquen directamente a la base de datos mediante vistas o DTOs, equilibrando así la balanza de rendimiento.

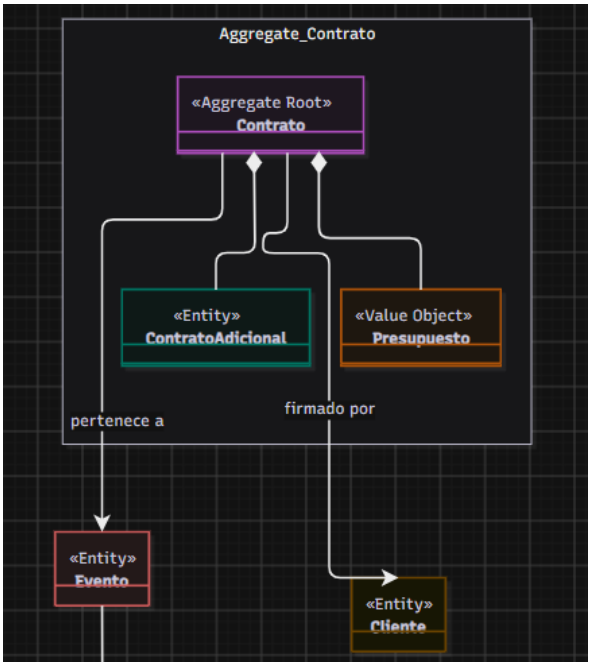
Etapa 3. Construir mapa de dominio

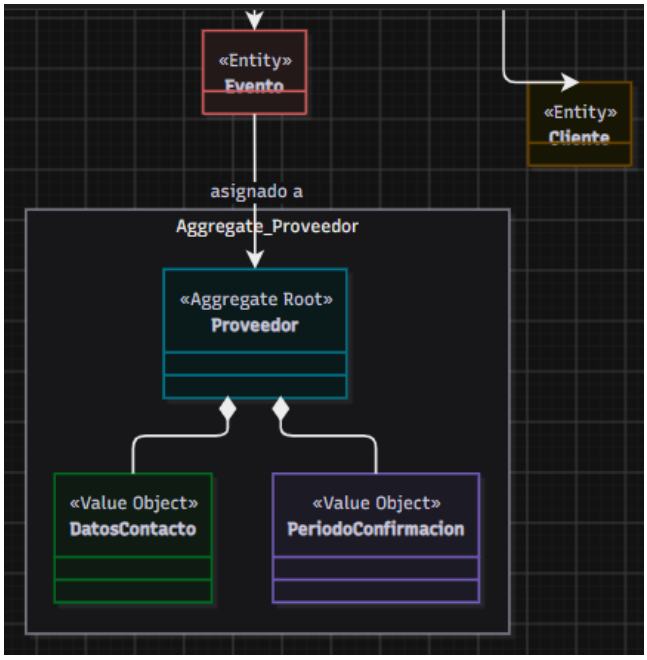
Siguiendo los principios de *Domain-Driven Design* (DDD) y respaldados por la elección del patrón *Domain Model*, en esta etapa se transforma el vocabulario del negocio y las reglas de la Unidad 2 en un mapa de responsabilidades estructurado. El objetivo es identificar y aislar los elementos que el sistema de Eventos Nova debe reconocer, proteger y comunicar. Se delimitan las fronteras de consistencia (Aggregates) para garantizar que ninguna operación viole las restricciones operativas o financieras, evitando convertir automáticamente simples tablas de datos en entidades. A continuación, se presenta la clasificación técnica de los componentes de nuestro dominio:

Elemento	Pregunta que debe responder el equipo
Lenguaje ubicuo	Términos clave compartidos por el equipo y el negocio: *Evento, Contrato Principal, Contrato Adicional, Proveedor, Límite de Cambios Normales, Costo Nova, Cargo Extra por Excedente, Lista de Advertencia, Anticipo Inicial.*
Entidades	Conceptos con identidad única (ID) que cambian con el tiempo: • Evento: Mantiene su identidad, aunque cambie de fecha, cantidad de invitados o estado. • Proveedor: Entidad externa registrada en el catálogo con un ciclo de vida propio (Activo, Bloqueado, En Lista de Advertencia). • Cliente: Persona o empresa registrada (Nombre, Teléfono, Correo).
Value Objects	Conceptos inmutables definidos por su valor (sin ID propio): • Presupuesto: Contiene el Monto y la Moneda. Regla: No puede ser negativo. • PeriodoConfirmacion: Un lapso (ej. 3 días) para la disponibilidad del proveedor. • DatosContacto: Agrupa Teléfono y Correo Electrónico. Si cambia un dígito, es un nuevo contacto.

Elemento	Pregunta que debe responder el equipo
Aggregates	Grupos que garantizan la consistencia (con una raíz): **1. Aggregate Root: Contrato** - *Responsabilidad:* Controlar el ciclo de vida financiero y las adiciones al evento. - *Elementos internos:* Contrato Principal, Contrato Adicional, Presupuesto. - *Invariantes:* No se agregan servicios si exceden el Presupuesto (US 11). Solo es "Activo" si se validó el Anticipo (US 05). **2. Aggregate Root: Proveedor** - *Responsabilidad:* Administrar disponibilidad, datos de contacto y estatus de confiabilidad. - *Elementos internos:* DatosContacto, PeriodoConfirmacion. - *Invariantes:* Si pasan 3 días sin respuesta, el estado cambia automáticamente a "Expirado" (US 06).
Domain Services	Operaciones complejas que no caben en una sola entidad: • PenalizacionService: Calcula montos de multa y coordina la cancelación en cascada de múltiples proveedores cuando el cliente no liquida tras el plazo de gracia de 2 días (US 13).
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes. • ContratoActivadoEvent: Se dispara cuando Administración valida el anticipo, notificando a los proveedores. • ProveedorCanceladoEvent: Detona la búsqueda de sugerencias de reemplazo y la inserción del proveedor en la Lista de Advertencia (US 07). • ServiciosModificadosEvent: Dispara notificaciones automáticas a los proveedores cuando el Coordinador autoriza un cambio (US 09).

Diagrama de mapa de dominio.





Nuevos términos al glosario.

Aggregate Root (Raíz de Agregado): Entidad principal (ej. Contrato) que actúa como el único portón de entrada para modificar los datos internos de su grupo, garantizando que no se rompan las reglas de Eventos Nova.

Presupuesto Operativo (Value Object): Representación inmutable del límite de dinero autorizado. No es solo una cifra, contiene la moneda y la lógica para saber si un nuevo servicio supera el límite.

Lista de Advertencia Activa: Estado específico dentro del ciclo de vida del Proveedor que se activa mediante un Evento de Dominio tras una cancelación.

Ficha de cada Aggregate.

Ficha Aggregate 1: Contrato Financiero

- Raíz (Root): Contrato
- Responsabilidades: Controlar el ciclo de vida financiero del evento, validar límites de presupuesto y aprobar la anexión de servicios extra.
- Invariantes (Reglas irrompibles):
    1. El costo total de ContratoAdicional + Contrato jamás puede exceder el Presupuesto autorizado sin autorización explícita.

2. Un contrato no pasa a estado "Activo" sin registrar el pago exacto del Anticipo Inicial (25%).

- Eventos Posibles: ContratoActivadoEvent, PresupuestoExcedidoEvent.

Ficha Aggregate 2: Gestión de Proveedores

- Raíz (Root): Proveedor
- Responsabilidades: Administrar los tiempos de confirmación, historial de fallas y datos de contacto de las entidades externas.
- Invariantes (Reglas irrompibles):
    1. Si no hay respuesta en el PeriodoConfirmacion (3 días), el estado cambia estrictamente a "Expirado".
- Eventos Posibles: ProveedorCanceladoEvent, AdvertenciaGeneradaEvent.

Explicación de las decisiones dudosas o descartadas

- **Decisión Dudosa 1:** Se debatió si un ContratoAdicional debía ser su propio Aggregate Root independiente, ya que el cliente puede solicitarlo días después.
- **Por qué se descartó:** Se decidió meterlo como una Entidad "hija" dentro del Aggregate Contrato porque un servicio extra no tiene razón de existir ni ciclo de vida propio si no está amarrado a un evento principal. Además, dejarlo suelto impediría calcular si la suma global excede el presupuesto (US 11).
- **Decisión Dudosa 2:** Convertir el "Contacto" (teléfono/correo) en Entidad.
- **Por qué se descartó:** Se modeló como *Value Object*. Si un cliente cambia su número de teléfono, no editamos el objeto "teléfono", simplemente destruimos ese dato y creamos uno nuevo, porque su identidad real radica en el Cliente, no en su número.

Etapa 4. Diseñar la Service layer.

Una vez estructurado el núcleo lógico del sistema en el Mapa de Dominio, el siguiente paso es diseñar la capa de aplicación (*Service Layer*). En esta etapa se definen los *Application Services* encargados de exponer los casos de uso operativos a los actores del sistema. El propósito estricto de estos servicios es funcionar de manera exclusiva como orquestadores: reciben la solicitud de entrada, recuperan los objetos necesarios mediante abstracciones, invocan el comportamiento del dominio, solicitan la persistencia y devuelven un resultado.

Para evitar el antipatrón de "modelo anémico" y respetar la decisión arquitectónica (ADR-001), se garantiza que esta capa de servicios no absorba ni acumule reglas de negocio. Toda validación temporal, cálculo o protección de invariantes es delegada a los *Aggregates* correspondientes. A continuación, se detallan los servicios propuestos y la justificación de sus responsabilidades:

Servicio	Método o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ContratoAppService	AprobarCotizacion() *(CU-01 / US 04)*	Guid contratoId, Guid coordinadorId	IContratoRepository, IUnitOfWork	CotizacionAprobadaDTO	Delega el cálculo exacto del 25% de anticipo a la entidad Contrato. El servicio no hace matemáticas.
ProveedorAppService	RegistrarCancelacion() *(CU-02 / US 07)*	Guid proveedorId, Guid eventoId	IProveedorRepository, IDomainEventDispatcher	CancelacionResultDTO	Delega la lógica de cambiar el estado interno a "Lista de Advertencia" a la entidad Proveedor
EventoAppService	SolicitarCambioNormal() *(CU-03 / US 08)*	Guid eventoId, DetalleCambioDTO	IEventoRepository, IUnitOfWork	bool (Éxito)	Delega la validación temporal (revisar si faltan menos de 10 días) a la entidad Evento

Servicio	Método o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ContratoAppService	AgregarServicioExtra() *(CU-04 / US 11)*	Guid contratoId, ServicioExtraDTO	IContratoRepository, IUnitOfWork	CostoActualizadoDTO	Delega el bloqueo financiero estricto (validar que Costo Total < Presupuesto) al Aggregate Root Contrato
AdminAppService	AplicarPenalizacion() *(CU-05 / US 13)*	Guid contratoId	IContratoRepository, PenalizacionDomainService	PenalizacionResultDTO	Delega la validación de los 2 días de gracia a la entidad Contrato y el cálculo de la multa al Domain Service.

Revisión obligatoria

- ¿El nombre del método expresa una acción del negocio?
  Sí. Se descartaron nombres técnicos como UpdateContrato() o PatchProveedor(). En su lugar, se utilizan intenciones claras del lenguaje ubicuo como AprobarCotizacion() y RegistrarCancelacion(), reflejando las acciones de los actores (Coordinador y Administración).
- ¿El método coordina o contiene decisiones que deberían estar en el dominio?
  Solamente coordina. Por ejemplo, en AgregarServicioExtra(), el servicio recupera el contrato de la base de datos y llama al método contrato.AgregarServicio(nuevoServicio). Si se excede el presupuesto, la entidad Contrato lanza una excepción, por lo que el Application Service está libre de sentencias lógicas y matemáticas.
- ¿El servicio depende de interfaces o de detalles concretos de base de datos?
  Depende 100% de abstracciones. Se inyectan colaboradores como IContratoRepository. El servicio no tiene ni idea de si los datos vienen de SQL Server, PostgreSQL o un archivo de texto, protegiendo así la capa de aplicación.
- ¿La operación tiene un inicio y un final transaccional comprensible?
  Sí. El Application Service inicia la transacción, ejecuta la acción del dominio y llama a IUnitOfWork.Commit(). Si el dominio lanza una alerta (ej. límite de 10 días violado), el servicio atrapa el error y la transacción jamás se confirma, asegurando consistencia.
- ¿Los errores del dominio pueden explicarse con términos del negocio?
  Totalmente. En lugar de regresar un error técnico como *"NullReferenceException"* o *"Error 500 DB FK"*, el dominio arrojará excepciones claras de negocio como

PresupuestoExcedidoException o CambioFueraDeTiempoException, las cuales se pueden mostrar directamente al Cliente o Coordinador.

Etapa 5. Separar persistencia y transacciones.

Para garantizar que el modelo de dominio permanezca completamente aislado de los detalles de infraestructura tecnológica (bases de datos, sentencias SQL o frameworks ORM), en esta etapa se implementa el patrón *Repository*. Se definen interfaces estrictas que actúan como colecciones en memoria, encargadas exclusivamente de recuperar y persistir los *Aggregate Roots* de Eventos Nova.

Asimismo, para asegurar la integridad de los datos, se define el alcance transaccional mediante el patrón *Unit of Work*. Este componente es el responsable de coordinar las operaciones que deben completarse en conjunto de forma atómica, definiendo claramente dónde inicia la transacción, el momento exacto de la confirmación (*Commit*) tras validar las reglas de negocio, y la ejecución de la reversión (*Rollback*) en caso de presentarse una excepción en el dominio. A continuación, se detallan las interfaces de persistencia propuestas:

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
IContratoRepository	Consultar y actualizar contratos, anticipos, presupuestos y penalizaciones.	ObtenerPorId(), Guardar()	Aggregate Contrato, ContratoAdicional y Presupuesto.
IProveedorRepository	Administrar proveedores, sus estados y cancelaciones.	ObtenerPorId(), Guardar()	Aggregate Proveedor, DatosContacto y PeriodoConfirmacion.
IEventoRepository	Consultar eventos y validar modificaciones según su fecha.	ObtenerPorId(), Guardar()	Entidad Evento, fecha, estado y cambios autorizados.

Etapa 6. Separar Commands y Queries

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	AprobarCotizacionCommand	contratoId, coordinadorId	AprobarCotizacionHandler	CotizacionAprobadaDTO	Cambia el estado de la cotización y calcula el anticipo del 25%
Command	AgregarServicioExtraCommand	contratoId, ServicioExtraDTO	AgregarServicioExtraHandler	AgregarServicioExtraHandler	Modifica el contrato y valida que no se exceda el presupuesto autorizado
Query	ConsultarContratoQuery	contratoId	ConsultarContratoHandler	ConsultarContratoHandler	Permite consultar el estado, presupuesto y anticipo sin modificar información
Query	ConsultarProveedoresQuery	eventoId	ConsultarProveedoresHandler	ListaProveedorDTO	Muestra los proveedores relacionados con un evento y sus estados para facilitar el seguimiento

Etapa 7. Revisar la propuesta con SOLID

Principio	Pregunta de revision	Evidencia que deben buscar
S	**Hallazgo (Buena decisión):** Los CommandHandlers (ej. AprobarCotizacionHandler) están separados de los Application Services.	**Impacto:** Cada clase tiene una sola razón para cambiar. Los Handlers solo enrutan la petición, y los AppServices coordinan. No se requirió cambio.
O	**Hallazgo (Buena decisión):** El uso de interfaces como IContratoRepository protege la lógica de negocio.	**Impacto:** El sistema está abierto a extensión (se puede cambiar SQL Server por MongoDB) pero cerrado a modificación (no se toca la Service Layer).
L	**Riesgo detectado:** Cargar objetos muy pesados en memoria (ej. el Evento con todos sus proveedores) para operaciones simples de lectura violaba la eficiencia esperada de las abstracciones.	**Cambio:** Se aplicó CQRS básico (Etapa 6). Ahora las Queries devuelven DTOs planos para lectura sin cargar los Aggregates pesados.

Principio	Pregunta de revision	Evidencia que deben buscar
I	**Hallazgo (Buena decisión):** En lugar de tener una mega-interfaz IRepositoryGenérico con docenas de métodos, se dividió en IContratoRepository, IEventoRepository, etc.	**Impacto:** Cada Application Service depende únicamente de los métodos que realmente necesita usar, evitando dependencias innecesarias.
D	**Hallazgo (Buena decisión):** El dominio y la aplicación no tienen dependencias directas de herramientas o bases de datos (SQL).	**Hallazgo (Buena decisión):** El dominio y la aplicación no tienen dependencias directas de herramientas o bases de datos (SQL).

Etapa 8. Diagrama de clases.

Con los componentes lógicos, de coordinación y de persistencia completamente definidos, esta etapa tiene como propósito consolidar y representar visualmente la arquitectura del sistema. A continuación, se presenta el Diagrama de Clases UML que integra y diferencia claramente las capas de Dominio, Aplicación e Infraestructura. Asimismo, se exponen los Diagramas de Secuencia que ilustran el comportamiento y la interacción de los objetos en los flujos operativos clave. Finalmente, se documentan las justificaciones técnicas definitivas que sostienen este diseño a través de los Registros de Decisiones de Arquitectura (ADRs).

Capa	Elementos incluidos	Responsabilidad	Etapa
Dominio	Aggregate Roots: Contrato, Proveedor. Entidades: Evento, Cliente, ContratoPrincipal, ContratoAdicional. Value Objects: Presupuesto, DatosContacto, PeriodoConfirmacion. Domain Service: PenalizacionDomainService. Domain Events y excepciones de negocio. Interfaces IContratoRepository, IProveedorRepository, IEventoRepository.	Proteger invariantes (presupuesto, límite de 10 días, anticipo de 25 %, tolerancia de 2 días, expiración de 3 días) y emitir hechos del negocio. Contiene las reglas; no conoce la base de datos.	3 y 5
Aplicación	Command Handlers y Query Handlers; Application Services (ContratoAppService, ProveedorAppService, EventoAppService, AdminAppService); interfaces IUnitOfWork, IDomainEventDispatcher, IContratoConsultas, IProveedorConsultas.	Recibir el caso de uso, obtener los Aggregates, invocar el comportamiento del dominio, delimitar la transacción y devolver un DTO o un error de negocio. No calcula ni decide reglas.	4 y 6
Infraestructura	ContratoRepository, ProveedorRepository,	Implementar las interfaces: traducir Aggregates a filas	5 y 6

	EventoRepository, SqlUnitOfWork, DomainEventDispatcher, ContratoConsultas, ProveedorConsultas.	(ORM o mapeadores), manejar la transacción y resolver las consultas de lectura mediante vistas o DTOs.	

Diagrama de clases (vista general).

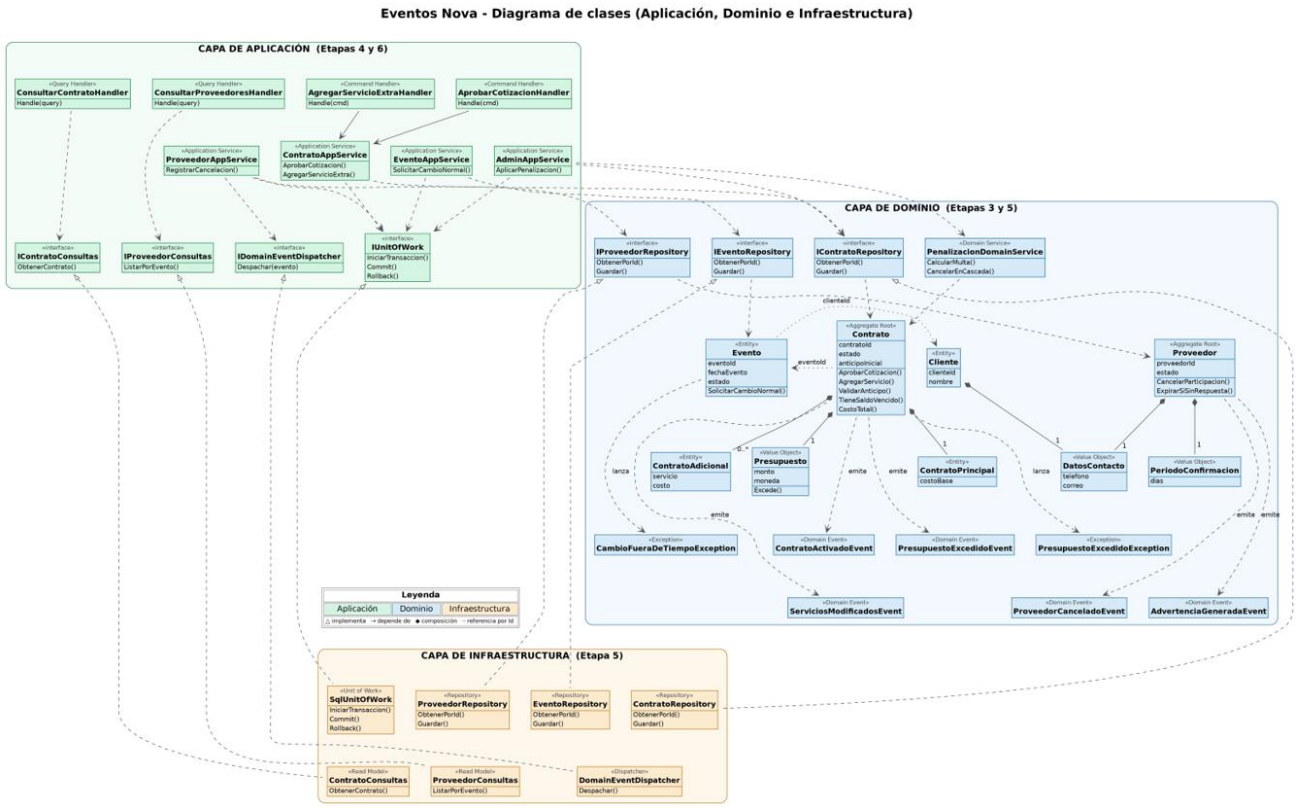


Figura 1A. Detalle de la capa de Dominio

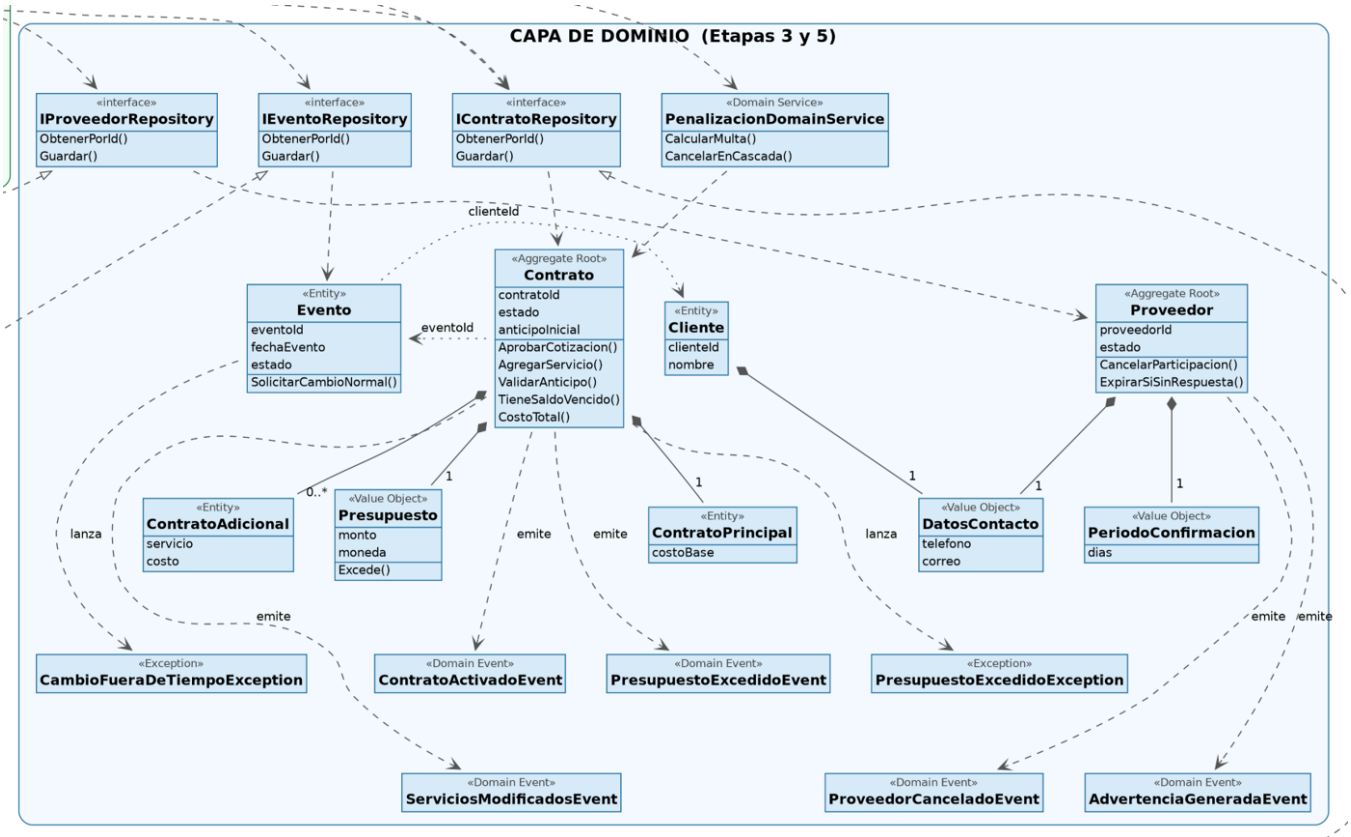


Figura 1B. Detalle de la capa de Aplicación

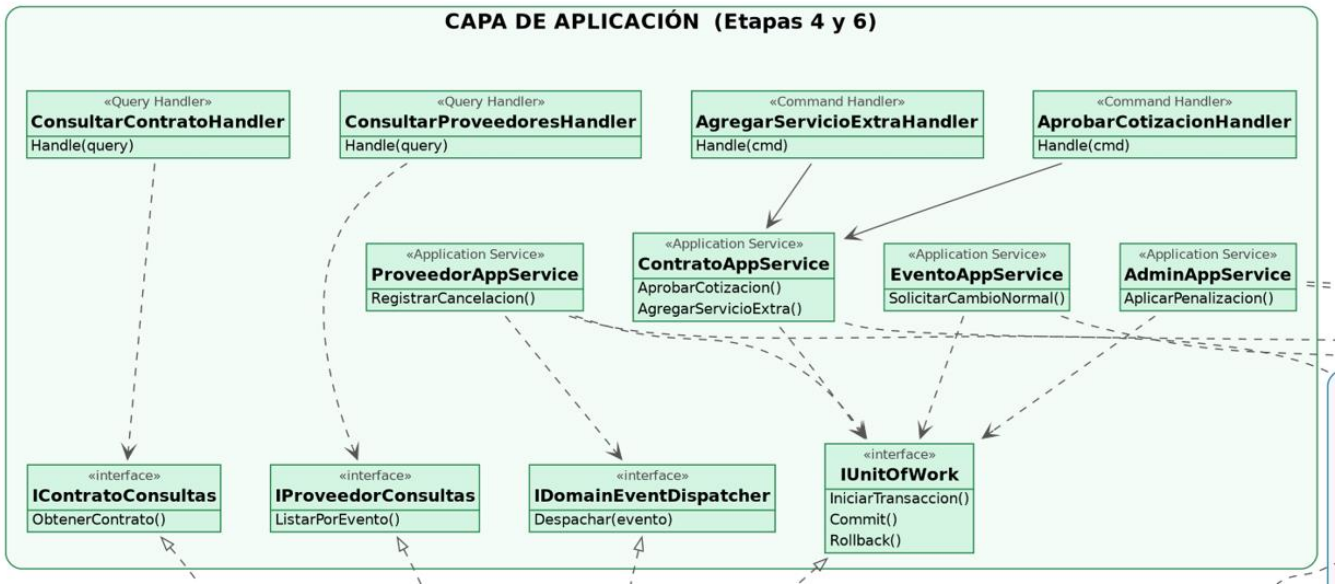
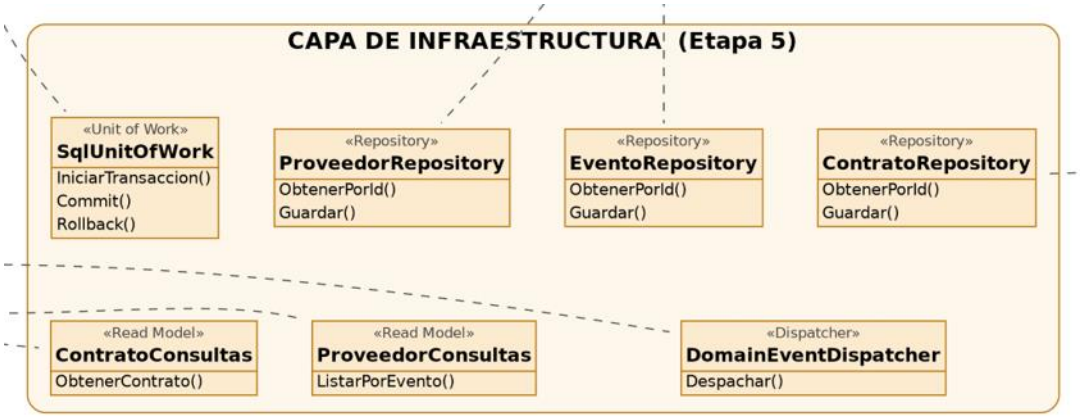
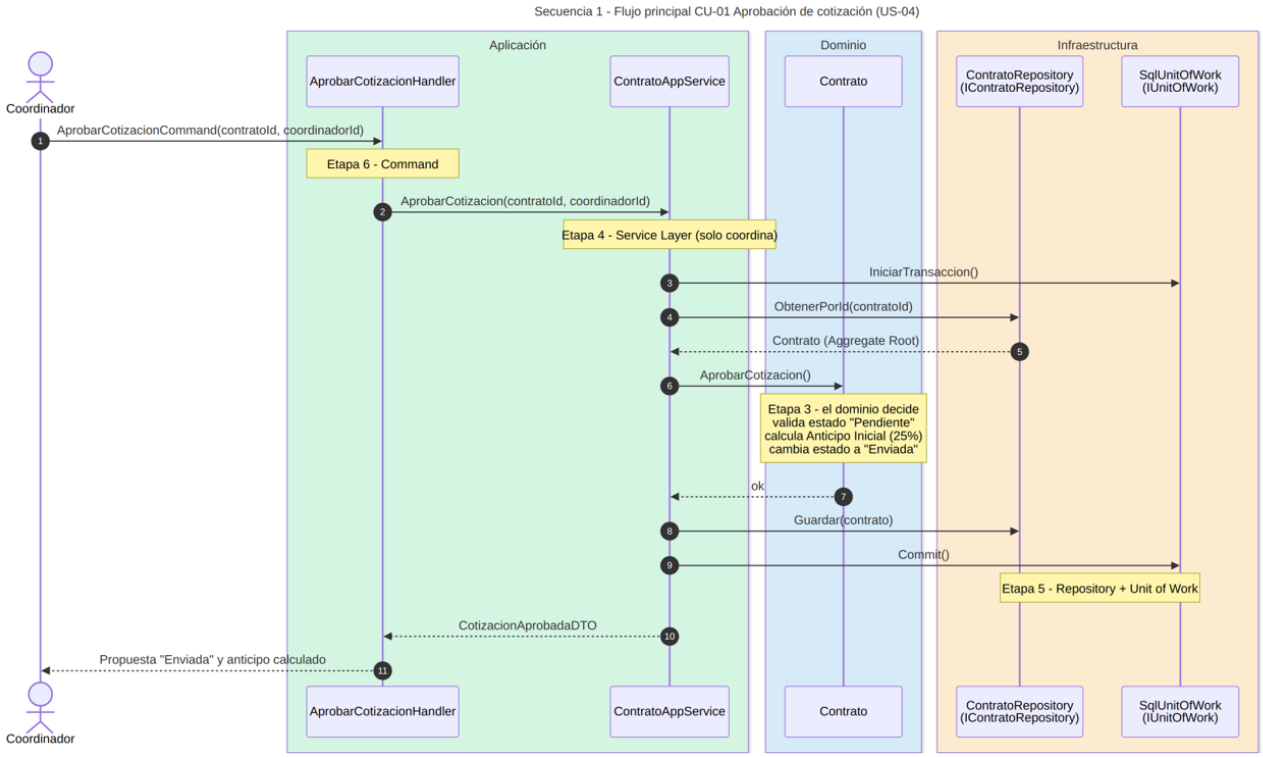


Figura 1C. Detalle de la capa de Infraestructura

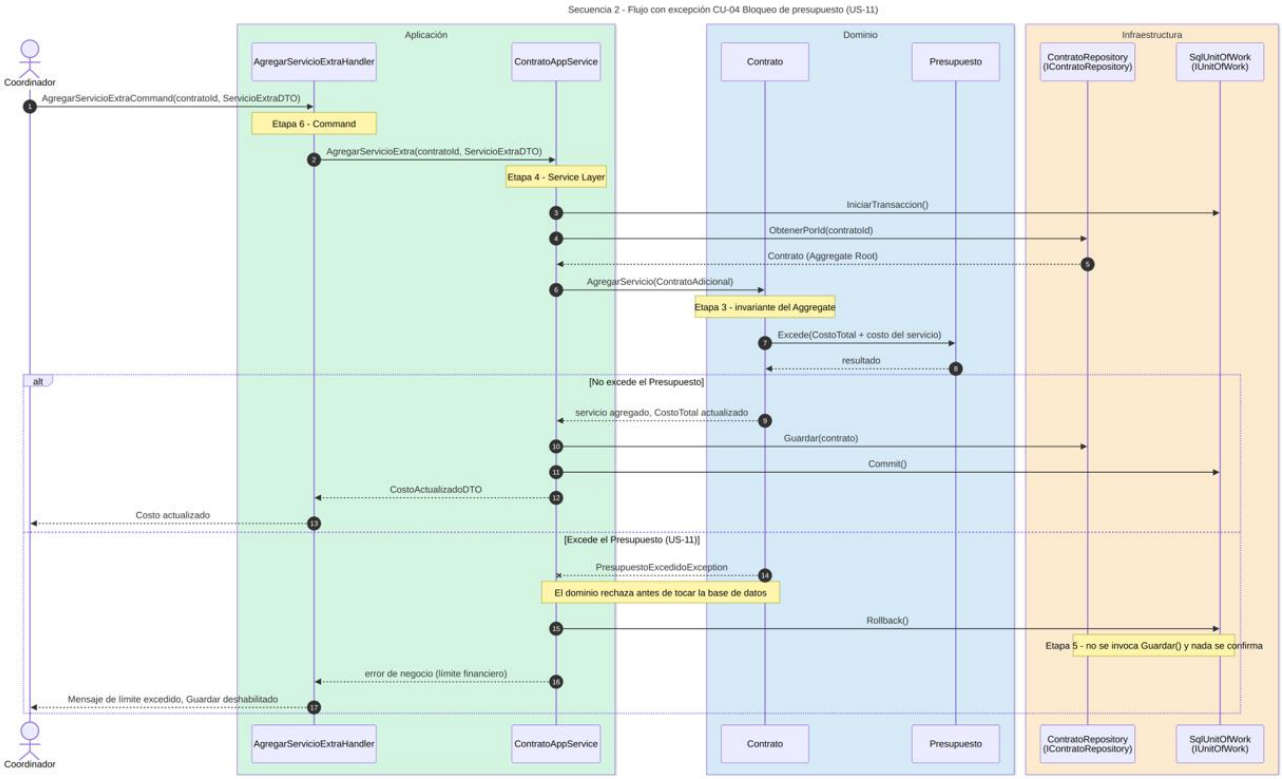


Diagramas de secuencia

Secuencia del flujo principal



Secuencia con excepción



Architecture Decision Records

ADR-1. Domain Model como patrón de lógica de negocio

Campo del ADR	Contenido
Título	Usar Domain Model como patrón principal de la lógica de negocio.
Estado	Aceptada.
Contexto	Eventos Nova depende de reglas financieras y temporales que interactúan entre sí: bloqueo de servicios adicionales que excedan el presupuesto (US-11), límite de 10 días para cambios normales (US-08), tolerancia de 2 días y cancelación en cascada por saldo pendiente (US-13), confirmación de proveedores en 3 días y lista de advertencia (US-07). Evidencia de U2: BRD (restricciones y necesidades del negocio) y Glosario (Contrato Activo, Límite de Cambios Normales).
Alternativas	Transaction Script: simple al inicio, pero dispersa los cálculos de fechas y penalizaciones en funciones sueltas y duplica reglas. Table Module: sirve para CRUD, pero la validación se complica cuando el límite depende del estado de otro registro (fecha del Contrato frente a fecha del Evento). Domain Model: concentra reglas e invariantes en objetos, a cambio de más código y mapeo.
Decisión	Domain Model. Las reglas viven en Contrato, Evento, Proveedor y PenalizacionDomainService, no en servicios ni en la base de datos.

Campo del ADR	Contenido
Consecuencias	Beneficios: invariantes protegidos en un solo lugar y reglas que pueden cambiar sin tocar la persistencia. Costos: curva de aprendizaje, mapeo objeto-relacional (ver ADR-004) y lecturas masivas costosas, mitigadas con ADR-005. Trabajo posterior: pruebas unitarias del dominio.
Trazabilidad	US-07, US-08, US-11, US-13; CU-02 a CU-05; BRD (restricciones); Glosario.

ADR-2. Contrato como Aggregate Root que incluye Contrato Adicional y Presupuesto

Campo del ADR	Contenido
Título	Agrupar Contrato Principal, Contratos Adicionales y Presupuesto bajo la raíz Contrato.
Estado	Aceptada.
Contexto	CU-04 exige comparar el costo total (contrato más adicionales) con el presupuesto autorizado, y el cliente puede pedir adicionales días después. Un servicio extra no tiene ciclo de vida sin un evento principal.
Alternativas	ContratoAdicional como Aggregate independiente: ninguna raíz podría validar el total y se rompería el invariante de US-11. Un único Aggregate con Evento y todos los proveedores: demasiado grande y costoso de cargar.
Decisión	Contrato es la raíz de un Aggregate con ContratoPrincipal, ContratoAdicional y Presupuesto. Evento y Proveedor quedan fuera y se referencian por Id.
Consecuencias	Beneficios: el invariante del presupuesto se verifica dentro de una sola transacción. Costos: toda modificación pasa por la raíz y el Aggregate crece con cada adicional. Trabajo posterior: decidir cómo manejar cambios concurrentes sobre el mismo contrato.
Trazabilidad	US-05, US-10, US-11; CU-04; Glosario (Contrato Principal, Contrato Adicional, Presupuesto Operativo).

ADR-3. Service Layer que solo coordina errores de dominio en lenguaje del negocio

Campo del ADR	Contenido
Título	Application Services sin reglas de negocio y excepciones de dominio expresadas en términos del negocio.

Estado	Aceptada.
Contexto	Cada caso de uso debe recibir la solicitud, cargar objetos, invocar al dominio, persistir y responder, sin que la Service Layer acumule reglas (Etapa 4). Los rechazos deben poder mostrarse al Coordinador o al cliente.
Alternativas	Reglas dentro de los servicios: se vuelve un Transaction Script disfrazado. Reglas en la interfaz: quedan expuestas y duplicadas. Servicios que coordinan y dominio que decide.
Decisión	Cuatro Application Services con métodos nombrados con intención de negocio (AprobarCotizacion, RegistrarCancelacion, SolicitarCambioNormal, AgregarServicioExtra, AplicarPenalizacion). El dominio lanza PresupuestoExcedidoException o CambioFueraDeTiempoException; el servicio hace Rollback y devuelve un error de negocio.
Consecuencias	Beneficios: reglas verificables sin infraestructura y nombres alineados con el glosario. Costos: más clases y conversión a DTO. Riesgo: que los servicios crezcan; se revisa en la Etapa 7 (SOLID).
Trazabilidad	CU-01 a CU-05; US-04, US-07, US-08, US-11, US-13.

ADR-4. Repository por Aggregate y Unit of Work en la capa de aplicación

Campo del ADR	Contenido
Título	Persistencia detrás de interfaces de Repository y transacción controlada por Unit of Work.
Estado	Aceptada.
Contexto	El dominio no debe depender de SQL ni del ORM, y las operaciones de cada caso de uso deben completarse juntas o no aplicarse (Etapa 5).
Alternativas	Acceso directo a la base de datos desde los servicios: acopla la aplicación a SQL. Repository genérico de CRUD: expone operaciones que el dominio no necesita. Repository por Aggregate con interfaces del dominio y un Unit of Work.
Decisión	IContratoRepository, IProveedorRepository e IEventoRepository (ObtenerPorId y Guardar) se definen junto al dominio y se implementan en infraestructura. IUnitOfWork (IniciarTransaccion, Commit, Rollback) lo invoca el Application Service. Ante una excepción de dominio se ejecuta Rollback y no se llama a Guardar.
Consecuencias	Beneficios: dominio sin SQL, base de datos intercambiable y límite transaccional claro. Costos: código de mapeo y sobrecarga del ORM. Trabajo posterior: definir si los Domain Events se despachan antes o después del Commit.

Trazabilidad	CU-01 a CU-05; hallazgo sobre IUnitOfWork en ProveedorAppService y AdminAppService.

ADR-5. CQRS básico sobre una sola base de datos

Campo del ADR	Contenido
Título	Separar Commands y Queries sin bases de datos distintas.
Estado	Aceptada.
Contexto	Cargar un Aggregate completo solo para leer datos es ineficiente, riesgo señalado en la Etapa 2. Las escrituras sí requieren pasar por el Domain Model.
Alternativas	Un solo modelo para lectura y escritura: las consultas cargan Aggregates completos. CQRS completo con bases separadas: complejidad desproporcionada para el caso. CQRS básico en la misma base de datos.
Decisión	AprobarCotizacionCommand y AgregarServicioExtraCommand pasan por su Handler hacia el Application Service y el dominio. ConsultarContratoQuery y ConsultarProveedoresQuery usan Handlers que leen mediante IContratoConsultas e IProveedorConsultas y devuelven DTOs de lectura, sin cargar Aggregates.
Consecuencias	Beneficios: lecturas ligeras y modelos de lectura independientes del de escritura. Costos: más piezas que mantener. Pendiente: justificar que CU-02, CU-03 y CU-05 sigan invocando directamente su Application Service.
Trazabilidad	CU-01 y CU-04; Etapa 2 (riesgo de lecturas masivas); Etapa 6.

Etapa 9. Validar la propuesta.

Validacion	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Los componentes propuestos están relacionados con los casos de uso CU-01 al CU-05 y las historias de usuario US-04, US-07, US-08, US-11 y US-13.

Validacion	Pregunta	Resultado del equipo
Lenguaje	¿Los mismos terminos significan lo mismo en BRD, UML, codigo y ADR?	Se mantienen términos como Contrato, Evento, Proveedor, Presupuesto y Anticipo Inicial para conservar la consistencia entre los documentos y la arquitectura.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patron elegido con una justificacion?	Las reglas de negocio permanecen en las entidades y servicios de dominio. Los Application Services únicamente coordinan las operaciones.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	La arquitectura propuesta mantiene relaciones entre entidades, repositorios y servicios. Los diagramas de clases y secuencia deberán representar estas mismas relaciones.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Se utilizan interfaces Repository para separar el dominio de la base de datos, evitando dependencias directas de SQL o herramientas externas.
Transaccion	¿El limite de la operacion y su manejo de error estan definidos?	Unit of Work permite confirmar las operaciones cuando se cumplen las reglas del negocio y revertir los cambios si ocurre algún error.
IA	¿Cada aporte de IA fue revisado y su uso esta registrado?	Se utilizó inteligencia artificial como apoyo para proponer y revisar la arquitectura. Las propuestas deben compararse con las reglas del negocio y registrar las correcciones realizadas por el equipo.

Bitácora de IA.

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Seleccionar patrón arquitectónico y delimitar 5 casos de uso clave	"Con base en las reglas del BRD de Eventos Nova, sugiere 5 Casos de Uso con flujos de excepción y justifica la elección de Domain Model."	Sugerencia de US 04, 07, 08, 11 y 13 con tabla comparativa a favor de Domain Model.	Error mínimo, la IA modifico ciertas palabras usadas en las users-stories	Se aceptaron las sugerencias de la IA, pero, manualmente se escribieron las user stories de acuerdo con el documento de la unidad 2
2	Construir el Mapa de Dominio identificando elementos clave.	"Revisa el Glosario y BRD de Eventos Nova. Propón Entidades, Value Objects, Aggregates (con sus invariantes) y Domain Events para la Etapa 3."	Se mapeó el "Contrato" como Aggregate Root para proteger las reglas de presupuesto y límites de días.	Supuesto: Se asumió que los "Contratos Adicionales" (US 10) viven dentro del Aggregate Root principal para validar el límite financiero global.	Aceptar la agrupación en el Aggregate Contrato para garantizar que la suma total no rompa el invariante del presupuesto autorizado.
3	Diseñar la Service Layer sin acumular reglas de negocio.	"Diseña los Application Services para los 5 Casos de Uso. El servicio solo debe coordinar. Justifica qué reglas delega al dominio y responde las preguntas de revisión obligatoria."	Se definieron firmas de métodos basados en intenciones (ej. AgregarServicioExtra) e inyección de dependencias mediante interfaces (Repositorios).	Supuesto: Se incluyó un IDomainEventDispatcher en la cancelación de proveedores para orquestar la lista de advertencia de forma asíncrona.	Se acepta el uso del Dispatcher para los Domain Events, manteniendo la Service Layer limpia de lógica procedimental

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
4	Generar Diagrama UML de Clases y revisión de principios SOLID.	"Analiza el diseño, señala hallazgos SOLID y genera el código PlantUML del diagrama de clases integrando Dominio, Aplicación e Infraestructura."	Se generó la tabla de los 5 hallazgos SOLID y el script de PlantUML con las flechas de dependencia invertida.	Supuesto: La IA asumió que los Handlers llaman a los AppServices directamente en vez del Dominio.	Aceptamos el modelo. Mantiene la Service Layer como punto central de orquestación transaccional y facilita la lectura del diagrama.
5	Evaluar la arquitectura completa bajo los cinco principios SOLID para detectar acoplamientos indebidos y corregir responsabilidades.	"Analiza el diseño actual de los repositorios y servicios de Eventos Nova y evalúa posibles violaciones a los principios SOLID, especialmente el principio de Responsabilidad Única (SRP) y Segregación de Interfaces (ISP)."	La IA identificó que una interfaz de repositorio monolítica obligaría a los servicios a depender de métodos que no usan, sugiriendo segregar los contratos por Aggregate Root y separar las consultas de lectura (IContratoConsultas) de los repositorios principales.	Supuesto: La IA sugirió modificar directamente las clases concretas de infraestructura para añadir lógica de caché.	Se rechazó modificar la infraestructura con lógicas ajenas y se aplicó estrictamente el principio de Inversión de Dependencias (DIP), asegurando que la capa de aplicación dependa exclusivamente de abstracciones e interfaces inyectadas.

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
6	Organizar y estructurar el repositorio de GitHub separando los entregables formales en directorios limpios (docs/, diagrams) para garantizar la mantenibilidad y correcta revisión del proyecto.	"Propón una estructura de directorios y un archivo README profesional para un repositorio de GitHub que albergue un proyecto bajo arquitectura limpia (Domain Model, Application, Infrastructure) para la empresa Eventos Nova."	La IA sugirió una jerarquía modular basada en carpetas estándar (docs para informes/bitácoras, diagrams para esquemas UML/PlantUML), además de un archivo README.md con instrucciones claras de despliegue y revisión.	Supuesto: La IA intentó generar scripts de automatización de despliegue (CI/CD con GitHub Actions) y archivos de configuración de Docker complejos que excedían el alcance académico de la actividad.	Se rechazó la inclusión de infraestructura de despliegue automatizado por innecesaria. Se conservó únicamente la estructura modular estricta de carpetas y el contenido documental requerido por la rúbrica institucional.

Enlace al repositorio de Github

https://github.com/cardenascarlostomas26-code/ACTIVIDAD_3_Componentes_de_logica_de_negocio/tree/main

Conclusión

El diseño arquitectónico desarrollado para el sistema de Eventos Nova demuestra una transición estructurada desde los requerimientos de negocio (establecidos en la Unidad 2) hacia una propuesta de software robusta, escalable y mantenible. Mediante la aplicación de los principios de *Domain-Driven Design* (DDD) y la rigurosa selección del patrón *Domain Model*, se logró encapsular la lógica de negocio compleja —tales como las restricciones financieras, los límites temporales y las penalizaciones— garantizando la protección absoluta de los invariantes críticos de la organización.

La separación de responsabilidades se mantuvo como un pilar fundamental durante todas las etapas del diseño. La estructuración de una capa de aplicación (*Service Layer*) puramente orquestadora, combinada con la abstracción de la base de datos a través de los patrones *Repository* y *Unit of Work*, asegura que el dominio permanezca tecnológicamente agnóstico. Asimismo, la integración de CQRS básico y la evaluación bajo los principios

SOLID certifican una arquitectura altamente cohesionada, con bajo nivel de acoplamiento y preparada para la evolución del negocio.

Finalmente, el uso documentado de la inteligencia artificial como herramienta de asistencia analítica permitió optimizar la validación del modelo y la toma de decisiones, dejando evidencia de un trabajo humano crítico y fundamentado. El resultado tangible —plasmado en diagramas UML precisos y Registros de Decisiones de Arquitectura (ADRs)— proporciona una base técnica sólida y coherente, lista para transicionar exitosamente hacia la fase de codificación y despliegue del sistema.